

Base de datos permanente para el RAG (auto-aprendizaje 100% persistente)

¿Por qué?

El filesystem de Render (plan free) es **efímero**: cada vez que el servicio se reinicia o se redespiega, se borran los archivos que no están en git. Eso significa que los ejemplos que el bot “aprende” automáticamente cada semana (guardados en `knowledge_base.json` / `embeddings_cache.json`) **se perderían** en el siguiente deploy.

La solución es guardar los ejemplos **y sus embeddings** en una base de datos Postgres externa. Así el aprendizaje sobrevive reinicios, redelloys y caídas.

✓ Ya está implementado en el código. Solo necesitas **crear la BD** y pegar la `DATABASE_URL` en Render. Si no la configuras, el sistema sigue funcionando en modo JSON local (como antes), sin romperse.

Cómo funciona (resumen técnico)

- Con `DATABASE_URL` **configurada** → modo **Postgres** (permanente):
 - Tabla `rag_examples` con los ejemplos + embeddings (columna `embedding` JSONB).
 - En el **primer arranque** con BD vacía, se siembra automáticamente desde `knowledge_base.json` (los 17 ejemplos curados).
 - Los embeddings se calculan **una sola vez** y se guardan. Tras un reinicio, NO se vuelven a calcular (ahorra costo de OpenAI y tiempo de arranque).
 - `POST /rag/learn` y el auto-aprendizaje semanal escriben directo en la BD.
 - Sin `DATABASE_URL` → modo **JSON local** (igual que antes):
 - Útil para desarrollo local y los tests. No es permanente en Render.
-

Opción recomendada: Neon (Postgres gratis y permanente)

[Neon](https://neon.tech) (<https://neon.tech>) ofrece un Postgres serverless con un free tier generoso y **permanente** (no expira como el de Render).

Pasos

1. Ir a <https://neon.tech> y crear una cuenta gratis (puedes usar GitHub/Google).
2. Clic en **“Create a project”**.
 - Nombre: `kaven-rag` (o el que quieras).
 - Región: la más cercana (ej. US West / US East).

3. Al crear el proyecto, Neon te muestra la **Connection string**. Cópiala.

Se ve así:

```
postgresql://usuario:password@ep-xxxx-yyyy.us-east-2.aws.neon.tech/neondb?sslmode=require
```

4. **Guárdala** — esa es tu `DATABASE_URL`.

Neon ya incluye `?sslmode=require`. Nuestro código usa SSL automáticamente con servidores gestionados, así que funciona tal cual.

Conectar la BD a Render

- 1. Entrar a <https://dashboard.render.com>
- 2. Abrir el servicio **kaven-bot**.
- 3. Ir a la pestaña **Environment** (Variables de entorno).
- 4. Clic en **“Add Environment Variable”**:
 - **Key:** `DATABASE_URL`
 - **Value:** (la connection string de Neon)
- 5. Clic en **Save Changes**. Render redesplegará automáticamente.
- 6. En los **Logs** de Render deberías ver:
 - ✓ RAG DB: Postgres conectado (persistencia permanente activa)
 - 🌱 RAG DB: sembrados 17 ejemplos desde `knowledge_base.json`
 - ✓ RAG listo: 17 ejemplos indexados (persistencia: pg)

¡Listo! A partir de aquí, todo lo que el bot aprenda queda guardado para siempre.

Alternativas a Neon

Proveedor	Free tier	Permanente	Notas
Neon ★	0.5 GB	✓ Sí	Recomendado. Serverless, se “duerme” pero no borra datos.
Supabase	0.5 GB	✓ Sí*	*Pausa proyectos inactivos tras 1 semana; se reactivan.
Render Postgres	1 GB	✗ No	Expira a los 90 días en free tier. No recomendado para esto.

Verificar / migrar manualmente (opcional)

El servidor siembra la BD solo en su primer arranque. Si quieres hacerlo a mano o **re-sincronizar** tras editar `knowledge_base.json` en el repo:

```
cd /ruta/al/repo/kaven-bot
DATABASE_URL="postgresql://...neon..." OPENAI_KEY="sk-..." node rag/migrate_to_db.js
```

Para inspeccionar lo que hay en la BD:

```
psql "postgresql://...neon..." -c "SELECT id, lang, sport, source, created_at FROM rag_examples ORDER BY created_at DESC LIMIT 20;"
```

Para ver el total:

```
psql "postgresql://...neon..." -c "SELECT COUNT(*) FROM rag_examples;"
```

Esquema de la tabla

```
CREATE TABLE rag_examples (
  id          TEXT PRIMARY KEY,          -- ej. kb-001 o kb-<timestamp>
  lang        TEXT DEFAULT 'es',         -- 'es' | 'en'
  sport       TEXT DEFAULT 'general',    -- ciclismo, futbol, baseball, general...
  tags        JSONB DEFAULT '[]',        -- ["catalogo","precio",...]
  outcome     TEXT DEFAULT 'positivo',   -- positivo | lead
  user_msg    TEXT,                      -- lo que escribió el cliente
  assistant_msg TEXT,                    -- la mejor respuesta
  note        TEXT,                      -- por qué funciona / contexto
  embedding   JSONB,                     -- vector del embedding (1536 dims)
  source      TEXT DEFAULT 'manual',     -- seed | manual | auto
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);
```

La columna `source` te permite distinguir el origen de cada ejemplo:

- `seed` → vino de los 17 ejemplos curados iniciales.
- `manual` → lo agregaste tú (vía `/rag/learn` o `load_meta_examples.py`).
- `auto` → lo aprendió solo la tarea semanal desde conversaciones reales.

Resumen

1. Crear Postgres gratis en **Neon** → copiar `DATABASE_URL`.
2. Pegarla en **Render** → **Environment**.
3. Render redespiega y siembra solo.
4. ☒ El auto-aprendizaje ahora es **100% permanente**.